

P1305R0

Deprecate The Addressof Operator

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1305r0>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1305](#)

Current Source:

slurps-mad-rips/papers/proposals/deprecate-addressof.bs

Abstract

The addressof operator (&`) might have had its use in being overloaded, however in practice countless issues have arisen from the simple possible case of "but what if someone overloaded it?". This brought about `std::addressof` which was first non-constexpr, then later made `constexpr`. Even if a minority of programmers do use it for representing some other memory model, why should the rest of the compiler vendors, C++ programmers, and library implementors have to make due with a function? It should be the other way around!

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 FAQ

3.1 Will this break code?

3.2 What's so bad about overloading the addressof operator?

4 Wording

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

To date, there have only ever been two widely used libraries that utilize overloading the addressof operator (&). These are the `ComPtr<T>` type from Microsoft and Boost.Spirit. Because of these two libraries (and anyone else who might get the bright idea to "just overload the addressof operator") standard library vendors are required to use `std::addressof`. This is, to be quite frank, ridiculous. In the same way that we have to protect our code from comma operator overloads (something that should also be deprecated and removed), we now have to protect ourselves from one of the most common operations when working with a systems programming language: Getting an object's address. While it might have made sense at one time to permit users to define their own memory model, this approach is no longer viable. Indeed, while `ComPtr<T>` overloads the addressof operator, it also provides a function that does the same operation. Additionally, Boost.Spirit uses it to represent a DSL from a different syntax altogether. While they might be hard pressed to replace these, they most likely don't need to get the address of the types they overload with.

Eventually removing the ability to overload the addressof operator will let us also remove `std::addressof` at some future date and compilers won't have to provide a `constexpr` builtin to do what is already built into the language.

§ 3. FAQ

§ 3.1. Will this break code?

No. We're only deprecating the `offsetof` operator. At some point in the future, it is assumed users will have migrated off of overloading the operator in favor of some function interface. At that point *then* we'll remove overloading the `offsetof` operator altogether.

§ 3.2. What's so bad about overloading the `offsetof` operator?

Although it is possible to do so in the language, one will note that the standard library at no point provides this overload itself (nor does it for `operator ,`, `operator ->*`, and others). This is by design and intent. These seemed like good ideas at the time, but it's 2018 now (as of this writing). Perhaps it's time for a little pragmatism.

§ 4. Wording

Wording is relative to [\[N4762\]](#)

In 11.3.1.2 Operators in expressions [**over.match.oper**], paragraph 3, add a new bullet point:

(3.4) — [\[Note: overloading the unary `operator &` is deprecated. See \[\\[depr.addressof\\]\]\(#\).\]](#)

In 11.5.1 Unary Operators [**over.unary**] add a new paragraph

³[\[Note: overloading the unary `operator &` is deprecated. See \[\\[depr.addressof\\]\]\(#\)](#)

In annex D, add a new paragraph

****Overloading unary operator & ** Overloading the unary operator & operator is deprecated. [Note: Using the builtin unary operator & is not deprecated — end note]. [Example:**

```
struct X {  
    Z* operator & (.) const; // deprecated  
};  
X x;  
Z* z = &x; // deprecated  
  
struct Y { };  
Y y;  
Y* w = &y; // Not deprecated
```

— end example].

§ References

§ Informative References

[N4762]

Richard Smith. [Working Draft, Standard for Programming Language C+](https://wg21.link/n4762). 7 July 2018. URL: <https://wg21.link/n4762>